ENGINEERING

Context Engineering for Production AI Agents: KV Cache, Prefix Caching, and Long-Context GPU Economics (2026 Guide)

← BACK TO BLOG • WRITTEN BY MITRASISH, CO-FOUNDER & CTO, SPHERON • PUBLISHED JUN 17, 2026 | UPDATED SEP 30, 2026

Context Engineering · Context Engineering AI Agents · KV Cache Hit Rate · Long Context Inference Cost · Prefix Caching · LLM Agent Infrastructure · RadixAttention · GPU Cloud · H200 · B200

Spheron
Context Engineering for AI Agents

Production agents are not compute-bound on generation. They are compute-bound on context. A ReAct agent making 10 tool calls in a session might produce 500 output tokens total and consume 800,000 input tokens, because each call carries the full system prompt, tool schemas, and conversation history. The model generation step takes milliseconds. The prefill pass over 80,000 input tokens takes seconds, and you pay for it on every call. For a broader look at how this fits the inference engineering discipline, see the inference engineering guide 2026.

What Context Engineering Is

Context engineering is the practice of deciding what goes into the context window, in what order, and how to cache and compress it to minimize prefill compute without hurting output quality.

It is distinct from prompt engineering, which focuses on what instructions and examples produce the best outputs. Prompt engineering optimizes quality. Context engineering optimizes cost and latency.

It is also distinct from RAG, which focuses on retrieval. RAG decides which external documents to fetch. Context engineering decides how to include those documents, whether to compress them, how to order them relative to cached prefixes, and whether the retrieval was even necessary given what's already in the KV cache.

Context engineering sits between the agent framework and the inference server. The framework decides what to run. The inference server handles computation. Context engineering determines how efficiently those two layers connect.

The 100:1 Input:Output Ratio

Standard chatbots have balanced input:output ratios, often 1:1 to 3:1. Agents are different. Every agent turn carries accumulated overhead:

- **System prompt**: 2K-10K tokens for instructions, persona, and constraints
- **Tool schemas**: 5K-50K tokens for function definitions passed with every request
- **Conversation history**: grows by 100-2,000 tokens per turn
- **Retrieved documents**: 5K-100K tokens per RAG call

A concrete example: a ReAct agent making 10 tool calls with a 2K system prompt, 50K tool schemas, and 30K conversation history at turn 10 sends roughly 82,000 input tokens per round and produces 500 output tokens. That's a 164:1 ratio. That input-heavy skew is exactly why agentic AI inference cost runs 5-30x higher than a chat exchange: the multiplier tracks re-sent input tokens, not generated output.

Workload	Typical input tokens	Typical output tokens	Ratio
Chatbot (short)	500	300	1.7:1
RAG pipeline	8,000	400	20:1
ReAct agent (early turns)	20,000	200	100:1
ReAct agent (late turns)	80,000	300	267:1
Code review agent	120,000	1,000	120:1

At 100:1 and above, input tokens dominate everything: cost, latency, and GPU utilization. The generating step is not the problem. For a deeper look at how memory layers (working memory, episodic memory, semantic memory) interact with token costs, the agent memory infrastructure guide covers those tradeoffs in detail.

KV Cache Hit Rate: The #1 Cost Lever

When the inference server processes input tokens, it computes key-value attention tensors for each token at each transformer layer. These tensors are stored in the KV cache. If a subsequent request shares the same prefix, the server can reuse those tensors instead of recomputing them. The fraction of input tokens served from cache rather than recomputed is the KV cache hit rate.

At 100:1 input:output ratios, prefill compute accounts for 85-95% of total GPU time per request. A 90% KV cache hit rate means the server skips 90% of that prefill work, reducing effective compute cost per request by 80-90%.

The math:

```
cost_saving = hit_rate * prefill_fraction * total_gpu_time
```

At 90% hit rate, 92% prefill fraction:

```
cost_saving = 0.90 * 0.92 = 82.8%
```

Cost comparison for a 70B model serving a workload with 80K input tokens per request:

KV cache hit rate	Effective prefill tokens	TTFT	Cost per 1M requests
0% (no prefix caching)	80,000	~8s	baseline
50%	40,000	~4s	~50% of baseline
90%	8,000	~800ms	~10% of baseline
95%	4,000	~400ms	~5% of baseline

The difference between 0% and 90% hit rate is not a marginal optimization. It is the difference between a $20,000/month GPU bill and a $2,000/month bill for the same workload. For the full KV cache memory math and per-layer VRAM calculations, see the KV cache optimization guide.

Prefix Caching and RadixAttention

Two implementations dominate production in 2026: SGLang's RadixAttention and vLLM's automatic prefix caching.

SGLang RadixAttention stores KV activations in a radix tree keyed by token sequence. When a new request arrives, SGLang walks the tree to find the longest matching prefix already in cache and starts computation at that branch point. Every request that shares a system prompt benefits. Every turn in a multi-turn conversation benefits more because each turn's history is the prefix for the next.

vLLM automatic prefix caching (APC) works at the block level. vLLM divides the KV cache into fixed-size blocks (16 tokens each by default) and hashes each block by its token content. Blocks from one request are reused in later requests if the hash matches.

Feature	SGLang RadixAttention	vLLM APC
Default on	Yes	vLLM V1 / v0.18.0+
Granularity	Token-level (radix tree)	Block-level (16 tokens/block)
Multi-turn handling	Strong (accumulates state per conversation branch)	Good
Monitoring metric	`sglang_cache_hit_rate`	`num_prefix_cache_hit_tokens`

To maximize cache hit rate from either:

1. Fix your system prompt character-for-character across requests. A single whitespace difference invalidates the entire cached prefix.
2. Fix tool definition order. Passing `[search, write, read]` in one request and `[read, search, write]` in another produces cache misses even though the schemas are identical.
3. Pass conversation history as a stable prefix before the new user turn. Don't regenerate or summarize turns unless you've flushed the cached context intentionally.
4. Keep retrieved documents in the same position relative to the system prompt. Interleaving retrieved chunks differently across turns fragments the prefix tree.

Workloads with 60%+ prefix overlap consistently hit 75-95% cache hit rates. Workloads where every request is unique (creative generation, personalized recommendations) see near-zero benefit. For the full deployment walkthrough including monitoring setup, see the SGLang production deployment guide.

Long-Context Infrastructure: VRAM From 32K to 1M Tokens

KV cache VRAM scales linearly with context length. For Llama 3.1 70B (80 layers, 8 GQA heads, 128 head_dim), the formula is:

```
kv_bytes = 2 * layers * kv_heads * head_dim * context_len * bytes_per_element
         = 2 * 80 * 8 * 128 * context_len * bytes_per_element
```

At BF16 (2 bytes per element), a single concurrent user at 128K context needs 42.9 GB for KV alone. Add 140 GB for model weights at FP16, and a single H200 (141 GB) cannot serve even one user at this context without FP8 KV quantization.

Context	BF16 KV per user	FP8 KV per user	Recommended GPU
32K	~10.7 GB	~5.4 GB	H100 SXM5 80 GB
128K	~42.9 GB	~21.5 GB	H200 SXM5 141 GB
512K	~171 GB	~85.5 GB	2x B200 SXM6 192 GB
1M	~343 GB	~172 GB	NVMe KV offload or sequence parallelism

These figures are for Llama 3.1 70B. Model architecture matters: different GQA configurations and head dimensions produce different KV footprints. Treat these as planning estimates, not guarantees.

FP8 KV quantization is the single most effective lever below 512K context. It halves KV memory with minimal quality impact on most models. Enable it in vLLM with `--kv-cache-dtype fp8` on H100 and H200 hardware.

Related deep-dives:

- AI memory wall and decode throughput limits as KV cache size grows
- Ring attention and sequence parallelism for contexts beyond 512K
- DeepSeek sparse attention for workloads where most tokens are sparse

Techniques for Context-Heavy Agent Workloads

Prefix and Prompt Caching

Enable prefix caching before anything else. It has the largest impact on cost and latency and requires nothing beyond a flag. The two terms in this heading aren't interchangeable once you're self-hosting: prefix caching is the vLLM/SGLang mechanism below, while "prompt caching" is what OpenAI, Anthropic, Gemini, and DeepSeek call the billing discount they wrap around that same mechanism on their own infrastructure. What is prefix caching, and how it differs from API-level prompt caching untangles the two and shows how to turn a hit rate into a GPU-count number for exactly this kind of agent workload.

For vLLM:

```bash
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --enable-prefix-caching \
  --kv-cache-dtype fp8 \
  --gpu-memory-utilization 0.92
```

For SGLang (RadixAttention is on by default, no flag needed):

```bash
python -m sglang.launch_server \
  --model-path meta-llama/Llama-3.1-70B-Instruct \
  --quantization fp8 \
  --enable-metrics \
  --host 0.0.0.0 --port 8000
```

To monitor hit rate with SGLang, scrape `sglang_cache_hit_rate` from the Prometheus `/metrics` endpoint. With vLLM, compute it as:

```
hit_rate = num_prefix_cache_hit_tokens / (num_prefix_cache_hit_tokens + num_prefix_cache_m
```

A well-configured agent workload should hit 70%+ after the cache warms across the first few requests.

The most common reason hit rates stay low: prompt variation. Even one character difference in the system prompt (trailing newline, different whitespace) causes a full miss. Audit your prompt templates before concluding that prefix caching doesn't work for your workload.

KV Offload to NVMe

When GPU HBM fills, you have two options: add more GPUs or tier cold KV blocks to NVMe. LMCache adds NVMe-backed KV persistence to vLLM.

Cold KV reads from a fast NVMe drive run at roughly 7 GB/s. Recomputing a 128K prefix on H100 takes about 11 seconds. Even at NVMe read speeds, serving from disk beats recomputing from scratch for most production workloads with large shared prefixes.

Setup:

```yaml
# lmcache.yaml
local_disk: true
local_disk_path: /mnt/nvme/lmcache
max_local_disk_size: 500 # GB
```

```bash
LMCACHE_CONFIG_FILE=lmcache.yaml python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --kv-transfer-config '{"kv_connector":"LMCacheConnectorV1","kv_role":"kv_both"}'
```

For a full NVMe KV offload walkthrough, see the NVMe KV cache offloading guide. For multi-node setups where you want to share a single KV cache pool across all server instances via Redis, see the LMCache vLLM deployment guide.

Sleep-Time Pre-Computation

For predictable context patterns, you can pre-warm the KV cache during idle GPU cycles rather than on the first live request. If you know that every user session will start with a 50K-token codebase context, compute and store the KV cache for that context during off-peak hours so the first request in a session hits the warm cache.

This is especially valuable for multi-tenant agents where each customer has a stable, large context (a codebase, a knowledge base, a document set) that rarely changes. One pre-computation pass per customer context, shared across all their requests. For the full sleep-time compute pattern including scheduling and cache invalidation strategy, see the sleep-time compute guide.

Measuring and Eliminating Context Waste

The average production agent context carries 40-70% dead weight: tool schemas for functions never called, conversation history from turns irrelevant to the current task, retrieved chunks that score high on retrieval similarity but low on actual relevance to the query.

Waste matters because every dead token contributes to prefill cost and KV cache pressure. At 100:1 ratios, a 30% reduction in input tokens reduces cost by roughly 30% and TTFT by roughly 30% with no model changes.

Identifying waste:

1. Token attribution analysis: run a small set of representative requests through the model and measure per-token attention weights at decode time. Tokens with near-zero attention across all decode steps are candidates for pruning.
2. Tool schema audit: log which tools are actually called per agent type. If a research agent never calls `write_file` in 10,000 sessions, remove that schema from its system prompt.
3. History compression threshold: track when older conversation turns stop influencing generation (roughly after 10-15 turns for most tasks). Summarize turns beyond that threshold with a small LLM rather than keeping raw text.

Removing waste:

- Compress historical turns using a fast small model (8B-class) into structured memory objects. The compressed summary is 10-50x smaller than raw history. See the agent memory guide for deployment patterns.
- Filter retrieved chunks by relevance threshold before injecting. A reranker with a score cutoff at 0.75-0.85 typically removes 30-50% of retrieved text with minimal impact on answer quality.
- Use dynamic tool schema injection: detect which tool category the current request targets and include only the relevant subset of schemas.

For the application-level semantic caching layer that intercepts repeated identical queries before they even reach the inference server, see the semantic cache guide.

Self-Hosted vs API: The Economics of Context-Heavy Agents

The standard advice is "use managed APIs for low volume, self-host at scale." The crossover point for context-heavy agents is lower than most teams expect.

At 100K input tokens per request and GPT-4o pricing (~$2.50 per 1M input tokens):

- 1,000 requests/month: $250
- 5,000 requests/month: $1,250
- 20,000 requests/month: $5,000

Spheron H200 runs $4.80/hr on-demand today. As a worked example at the $4.84/hr rate recorded on 17 Jun 2026, a dedicated H200 ran $3,484/month. At 50% utilization with FP8 quantization and prefix caching, that instance handles roughly 2-4 million 100K-token requests per month, depending on concurrency and hit rate. Per-request cost drops below $0.001.

Monthly volume	GPT-4o cost (100K input/req)	H200 on Spheron ($4.84/hr on 17 Jun 2026, 50% util)	Cheaper option
1,000 req	$250	$3,484 (fixed)	API
5,000 req	$1,250	$3,484 (fixed)	API
20,000 req	$5,000	$3,484 (fixed)	Self-hosted
100,000 req	$25,000	$3,484 (fixed)	Self-hosted

The crossover is roughly 14,000 requests per month at 100K tokens each, assuming 50% GPU utilization. At higher utilization, the crossover drops further. At lower context lengths, it shifts higher.

For workloads requiring 512K+ context, B200 is the right tier. At the $7.41/hr per-GPU B200 rate Spheron listed on 17 Jun 2026 (B200 is $9.80/hr on-demand today), two B200s (192 GB each, 384 GB total) ran $10,670/month and can serve 512K-context agents at meaningful concurrency. At that context length, managed APIs become extremely expensive: 500K tokens at $2.50 per 1M = $1.25 per request. Around 8,600 requests/month exceeds the B200 pair cost.

> Pricing fluctuates based on GPU availability. The worked examples above freeze Spheron rates recorded on 17 Jun 2026; the live Spheron rates quoted are current as of **18 Oct 2026**. API prices reflect their most recent published rates and may have changed. Check current GPU pricing → for live rates.

Spheron H200 and B200 for Context-Heavy Agents

H200 SXM5 has 141 GB of HBM3e memory at 4.8 TB/s bandwidth. With FP8 quantization applied to both model weights and KV cache, a 70B model shrinks to roughly 70 GB, leaving around 71 GB for KV cache. That fits 3 concurrent users at 128K context each (about 21 GB of KV per user at FP8), or 1 user at 256K. FP8 KV cache quantization alone is not sufficient here: without also quantizing the model weights, the 70B model weights alone occupy around 140 GB at FP16, leaving no room for KV cache on a 141 GB GPU. That covers most production agentic workloads where concurrency per instance stays in single digits.

B200 SXM6 has 192 GB of HBM3e at 8 TB/s bandwidth. Two B200s give 384 GB total, which fits a 70B model plus KV cache for 7 concurrent users at 256K context each at FP8. The 8 TB/s bandwidth matters specifically for decode throughput when serving large concurrent batches where KV cache reads dominate memory access patterns.

Both GPU tiers on Spheron are bare-metal instances with NVMe local storage, which matters for LMCache KV offload. Managed GPU services that abstract away the underlying hardware often prevent direct NVMe access, limiting KV offload to CPU RAM (which maxes out around 200 GB/s versus 7 GB/s for NVMe, but also fills up faster under sustained load).

Per-minute billing means you pay for compute time, not reserved capacity. For agent workloads with uneven traffic, this matters: a burst hour costs the same as the first minute of a non-burst hour, not a full reserved hour. For how H200 and other agent-tier GPUs price out against hyperscalers on a per-completed-task basis, see AI agent hosting: best GPU cloud for cost and latency.

Starting points for H200 and B200 on Spheron:

- H200 on Spheron for 70B agents at 32K-256K context
- B200 on Spheron for large models or high concurrency at 256K+ context

---

> Context-heavy agents need high-VRAM GPUs with KV-cache-friendly serving stacks. Spheron's H200 and B200 instances come bare-metal with NVMe storage, vLLM/SGLang pre-configured, and per-minute billing so you pay for compute, not idle time.
>
> H200 on Spheron | B200 on Spheron | View all GPU pricing

STEPS / 06

Quick Setup Guide

01 **Profile your agent's input:output token ratio**

Add token logging to your agent loop. Record input_tokens and output_tokens per LLM call. If your ratio exceeds 10:1 (input to output), context engineering will have a larger impact on cost than any model-level optimization. If the ratio exceeds 50:1, context cost is the dominant factor and prefix caching is your highest priority.

02 **Enable prefix caching in vLLM or SGLang**

For vLLM: add --enable-prefix-caching to your launch command (enabled by default in vLLM V1/v0.18.0+). For SGLang: RadixAttention is on by default. Critical: fix your system prompt byte-for-byte across calls - any whitespace change invalidates the cached prefix. Fix tool definition order in every request.

03 **Instrument KV cache hit rate**

For SGLang: enable --enable-metrics and scrape the sglang_cache_hit_rate Prometheus metric. For vLLM: query the /metrics endpoint and track num_prefix_cache_hit_tokens / (num_prefix_cache_hit_tokens + num_prefix_cache_miss_tokens). A healthy agent workload should achieve 70%+ hit rate after the first few requests warm the cache.

04 **Calculate VRAM requirements for your context window target**

Use: kv_bytes = 2 * num_layers * num_kv_heads * head_dim * context_length * bytes_per_element. For Llama 3.1 70B at FP8: 2 * 80 * 8 * 128 * context_len * 1 bytes. At 128K context: ~21.5 GB per concurrent user slot. For 10 concurrent users at 128K: 215 GB KV cache, requiring at least 3x H200 (141 GB each) for KV alone plus model weights.

05 **Configure NVMe KV offload for >128K contexts**

Install LMCache: pip install lmcache. Create lmcache.yaml with local_disk: true, local_disk_path: /mnt/nvme/lmcache, max_local_disk_size: <available NVMe in GB>. Launch vLLM with: LMCACHE_CONFIG_FILE=lmcache.yaml python -m vllm.entrypoints.openai.api_server --kv-transfer-config '{"kv_connector":"LMCacheConnectorV1","kv_role":"kv_both"}'. Cold KV reads from NVMe (~7 GB/s) still beat recomputing a 128K prefix (~11s on H100).

06 **Set up LMCache for multi-node KV cache sharing**

Start Redis: docker run -d -p 6379:6379 redis:7. Add remote_url: redis://YOUR_REDIS_IP:6379 to lmcache.yaml. With a shared Redis backend, all vLLM workers across nodes share the same prefix cache pool. The first worker that computes a given prefix writes it to Redis; every subsequent worker on any node serves it from cache.

FAQ / 05

Frequently Asked Questions

01 **What is context engineering for AI agents?** ×

Context engineering is the practice of designing and managing what goes into an AI agent's context window to minimize compute cost without degrading output quality. In 2026 agents routinely send 50,000-500,000 input tokens per request against only a few hundred output tokens. Because LLM APIs price input and output tokens differently, and because compute cost scales with context, deciding what to include, cache, compress, or prune is the biggest cost lever in agentic AI systems.

02 What is KV cache hit rate and why does it dominate agent inference costs? +

03 How much VRAM is needed for 128K to 1M token context windows? +

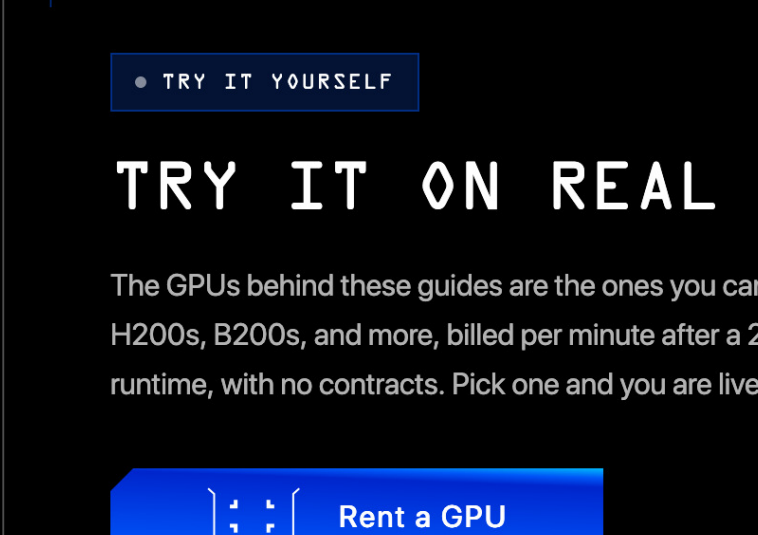
04 What is RadixAttention and how does prefix caching reduce inference cost? +

05 When does self-hosted GPU beat per-token API pricing for context-heavy agents? +

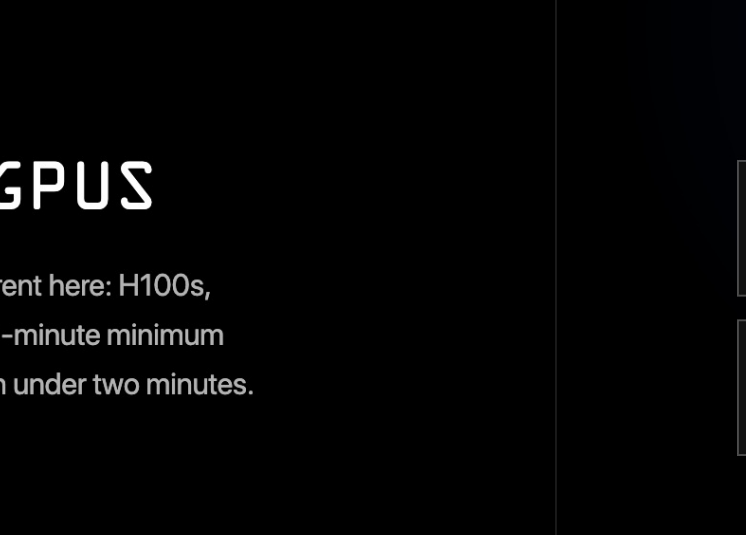
← Back to all posts

Related Reading

MORE ON INFERENCE OPTIMIZATION →


PILLAR GUIDE
Spheron
How Much VRAM Do You Need for LLMs?
7B to 70B+ Models • FP16, INT8, INT4

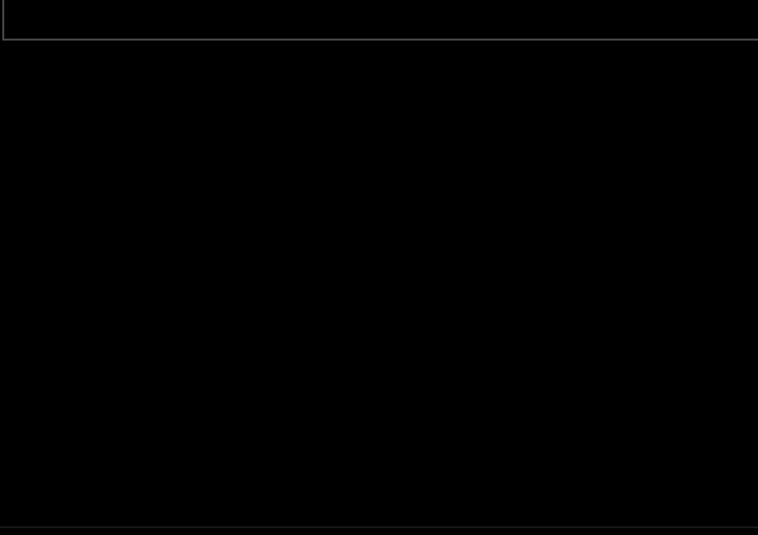
LLM VRAM Requirements: How Much GPU Memory You Need (2026)
May 15, 2026



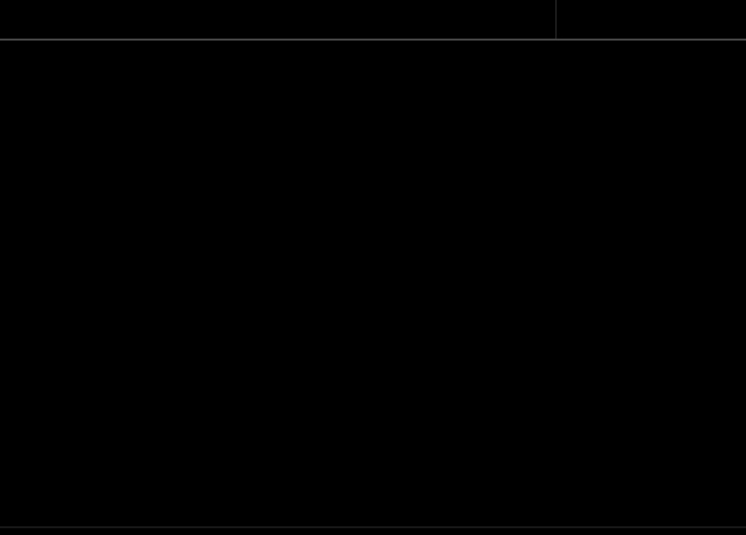
NVIDIA ICMSP Explained: KV Cache NVMe Offload, 5x Inference Gains, and Setup Guide (2026)
Jun 6, 2026

ENGINEERING
Spheron
Inference-Time Compute Scaling
Reasoning Models KV Cache & GPU Sizing

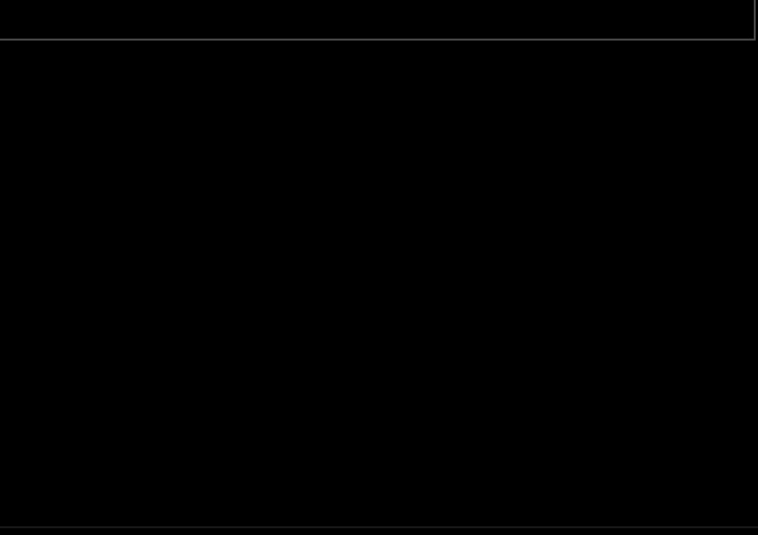
Inference-Time Compute Scaling on GPU Cloud: Allocate More GPU to Think Harder, Not Train Bigger...
Apr 7, 2026



Token Cost Scaling Context Window: The 128K Bend (2026)
Sep 19, 2026



NVIDIA Transformer Engine: FP8 Mixed Precision on H100 and H200 (Setup, Code, Benchmarks 2026)
May 22, 2026



MiMo-V2.6-Pro GPU Requirements: 1T MoE Setup Guide (2026)
Oct 4, 2026

• TRY IT YOURSELF

TRY IT ON REAL GPUS

The GPUs behind these guides are the ones you can rent here: H100s, H200s, B200s, and more, billed per minute after a 20-minute minimum runtime, with no contracts. Pick one and you are live in under two minutes.

Rent a GPU

DEPLOY TIME < 2 min

UPTIME SLA 99.9%

GPU MODELS 10+

STARTING AT $0.59/hr

GLOBAL COMPUTE, BROUGHT TO YOU BY
Spheron
Bare-metal GPU infrastructure for AI teams that need scale, speed, and simplicity.

Deploy Now
Contact Sale →

PRODUCT
App
Pricing
Customers
GPU Catalog
Supply GPU

RESOURCES
Documentation
API Reference
Blog
GitHub
Changelog

TOOLS
GPU Recommender
Training Cost Calculator

FEATURES
On-Demand Instances
Spot Instances
Reserved Commitments

COMMUNITY
Discord
X (Twitter)
LinkedIn
YouTube
Telegram

GPUS
NVIDIA Vera Rubin
NVIDIA GB300
NVIDIA GB200
NVIDIA B300
NVIDIA H100
NVIDIA B200
NVIDIA H200
NVIDIA A100
NVIDIA GH200
NVIDIA L40S
AMD MI300X
RTX 4090
RTX 5090
RTX PRO 6000

© 2026 SPHERON. ALL RIGHTS RESERVED.